Project Overview

The AWS Translator Web Application integrates multiple AWS services to deliver automated, multi-language translation.

- The project illustrates the use of **AI, Natural Language Processing (NLP), and Serverless Computing** within AWS Cloud.
- Initially developed using **Amazon Lex** and **AWS Lambda**, and later optimized to a **frontend-only architecture** using **Cognito**, **Translate**, and **Comprehend**.

Objectives

Multi-Language Translator

Design a **multi-language translator** capable of automatic language detection.

AWS Integration

Demonstrate **integration between AWS AI and serverless services**.

Static Deployment

Deploy a **fully static web application** using AWS Free Tier resources.

Secure Experience

Provide a **secure, cost-effective, and low-latency translation experience** directly through the browser.

AWS Services Utilized

Below is a list of the AWS services employed in this project and their respective purposes:

	Amazon Translate	Performs language translation.
	Amazon Comprehend	Detects source language automatically.
	Amazon Cognito	Provides temporary credentials for secure browser access.
	Amazon S3	Hosts static frontend resources.
	AWS IAM	Manages access policies and user roles.
	Amazon Lex (Phase 1)	Enables chatbot-based translation interface.
	AWS Lambda (Phase 1)	Executes translation logic through Python code.
	Amazon CloudWatch	Monitors logs and performance metrics.
	AWS CloudShell	Used for CLI testing, resource creation, and debugging.

(Official References: AWS Translate, Comprehend, Cognito, S3, Lex, Lambda, CloudWatch, CloudShell, IAM)

Architecture: Phase 1(Chatbot-Based System)

This initial phase focuses on a chatbot-driven translation interface.

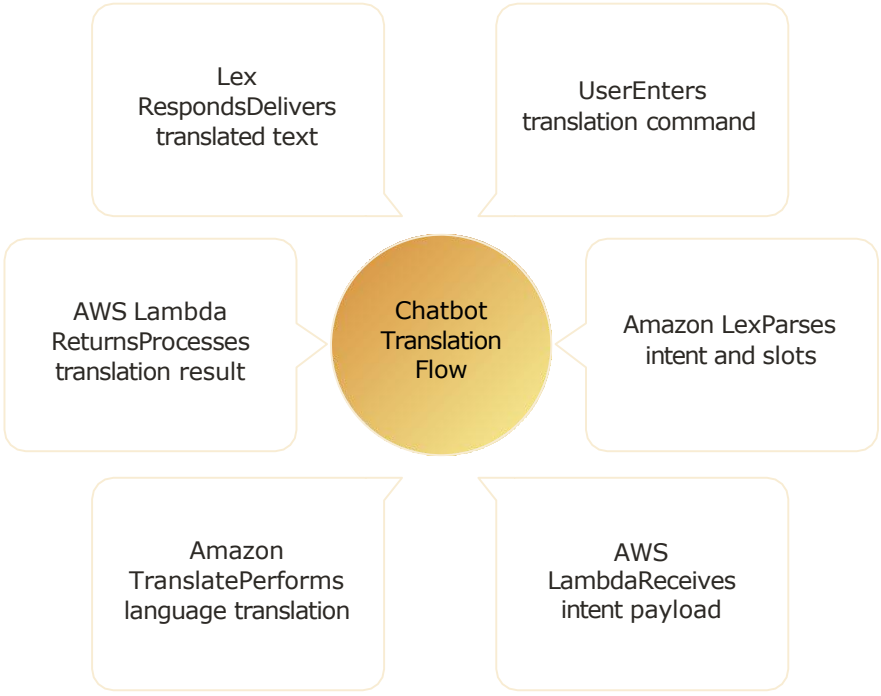
Workflow:

- User inputs a translation command (e.g., "Translate hello to French") through **Amazon Lex**.
- **Lex** identifies the intent and triggers a **Lambda** function.
- **Lambda** invokes **Amazon Translate** for translation and returns the response via Lex.

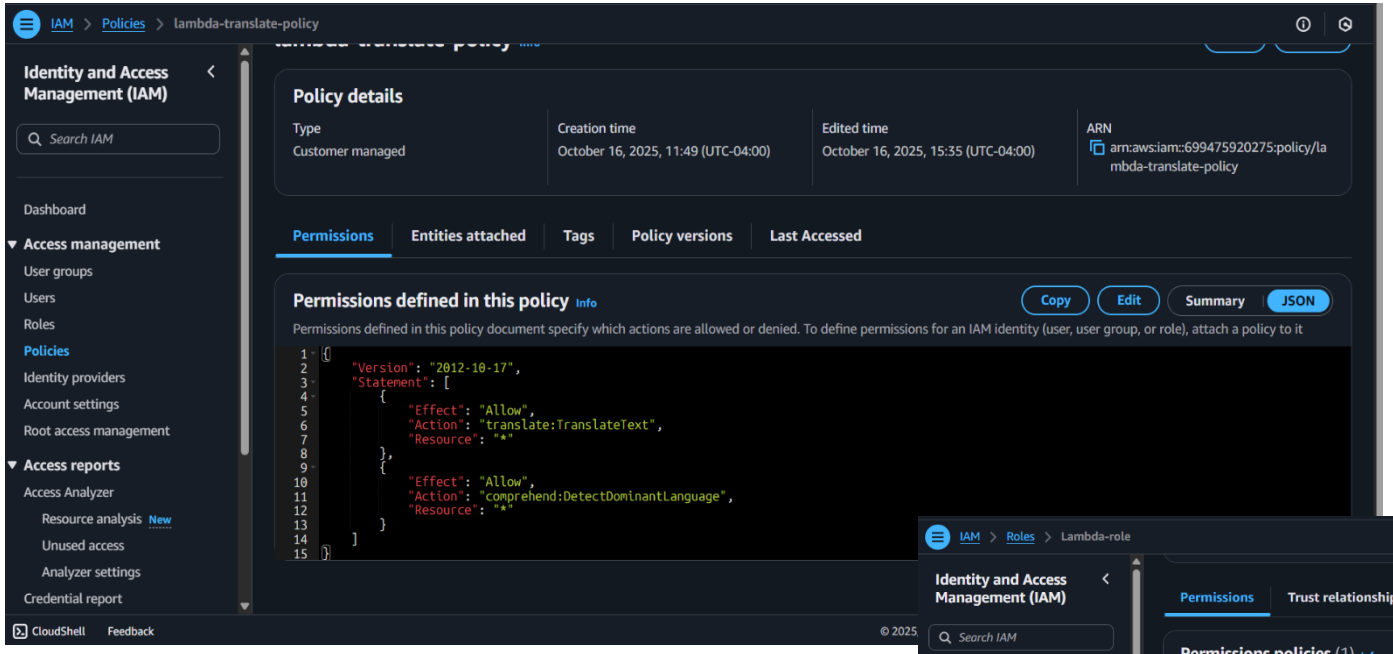
Flow Diagram: User → Lex Bot → Lambda → Amazon Translate → Lex → User

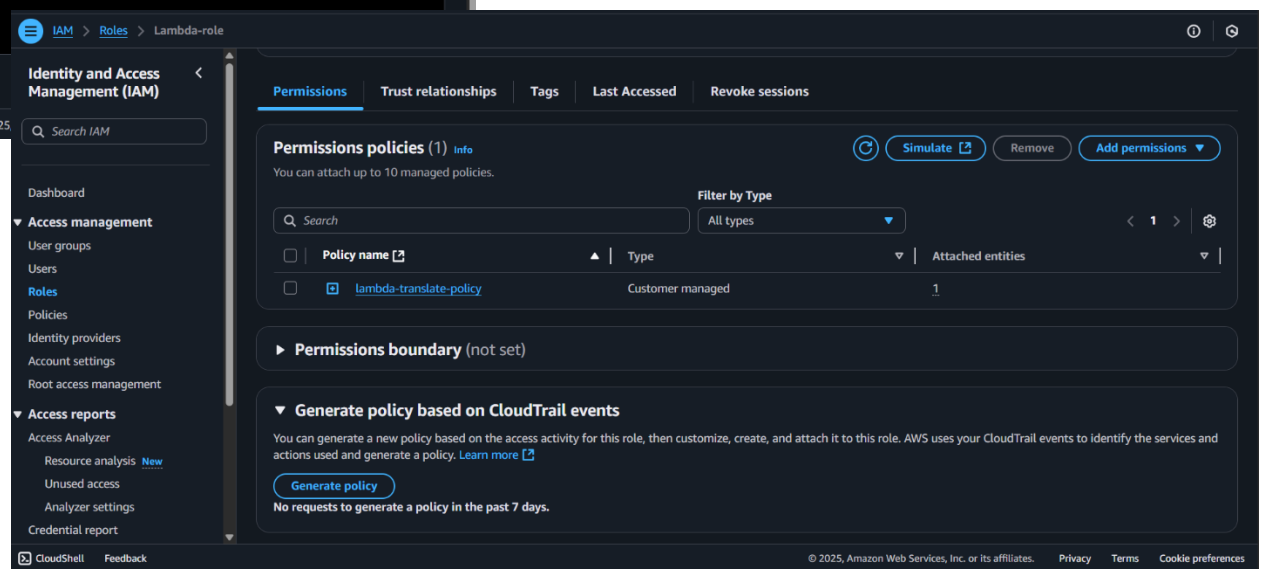
Highlights:

- Demonstrates **Natural Language Understanding (NLU)** using intents and slots.
- Showcases integration of multiple AI services.
- Higher latency due to multi-step invocation chain.

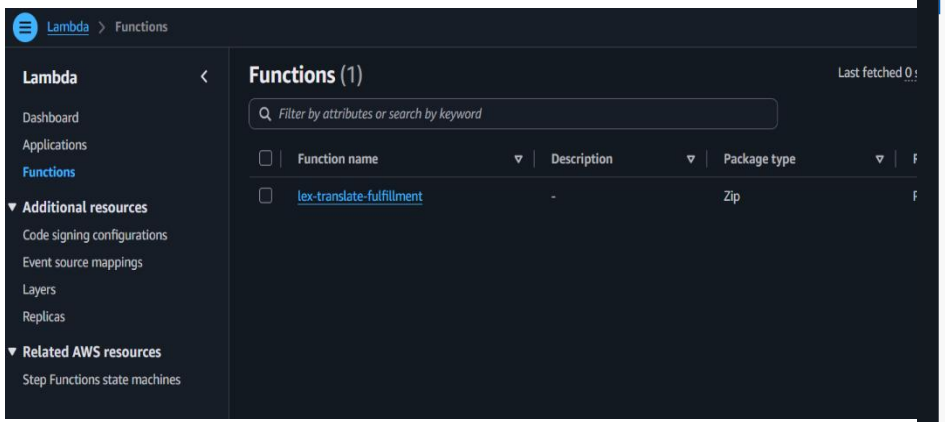


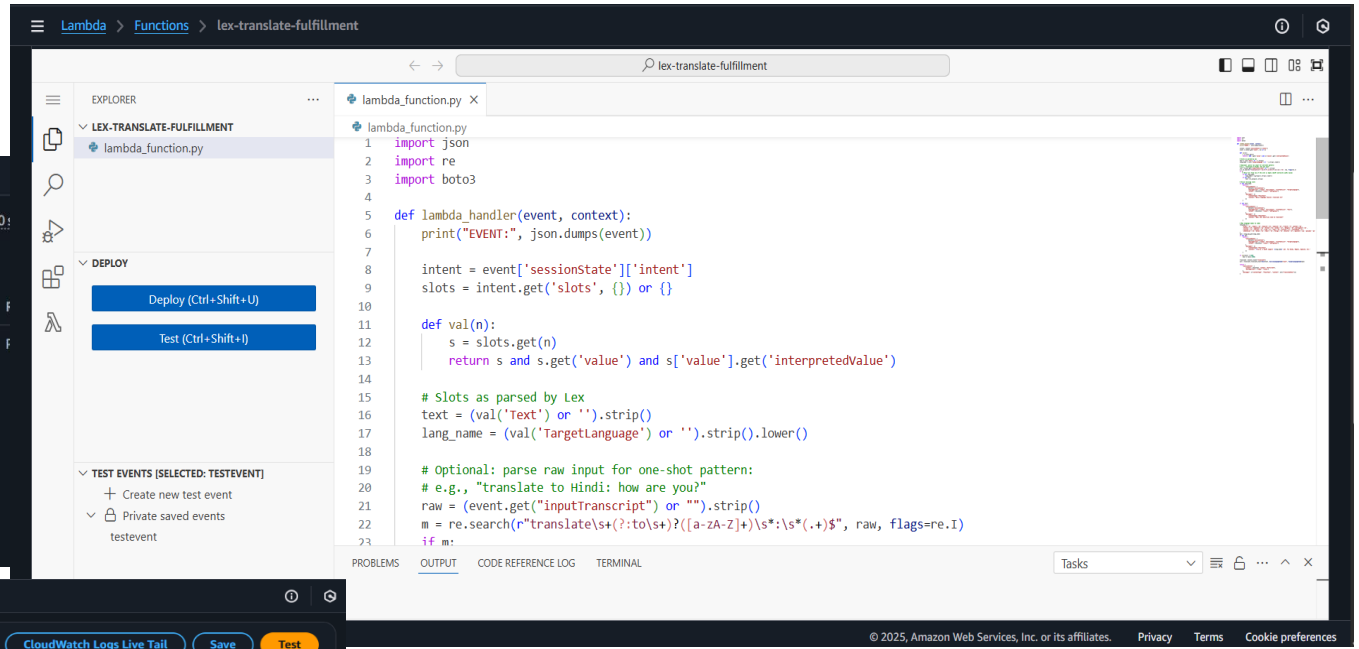
IAM Roles and policies for phase 1

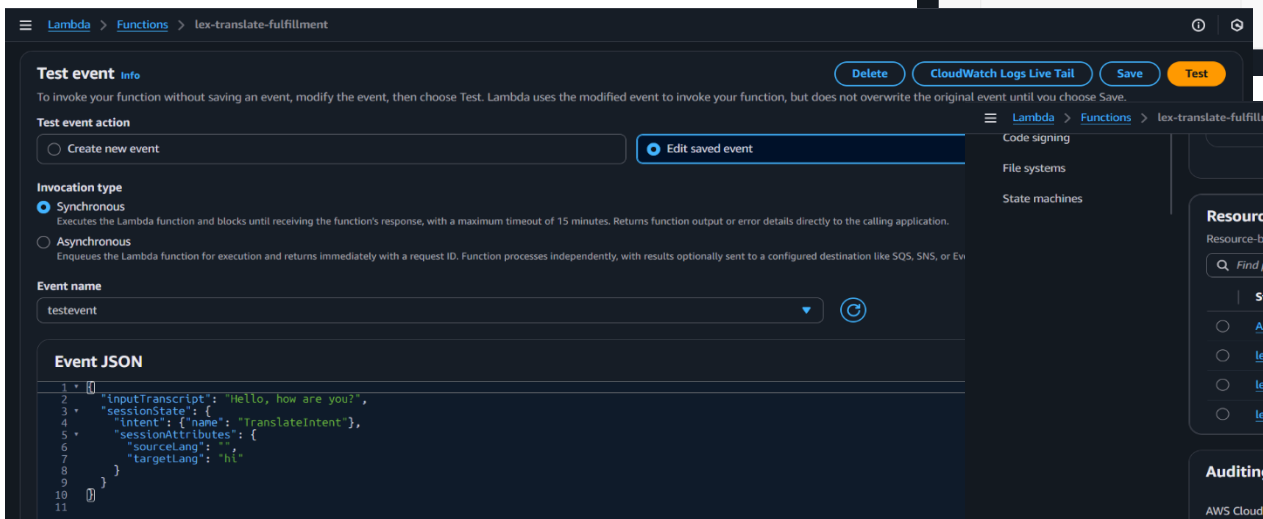


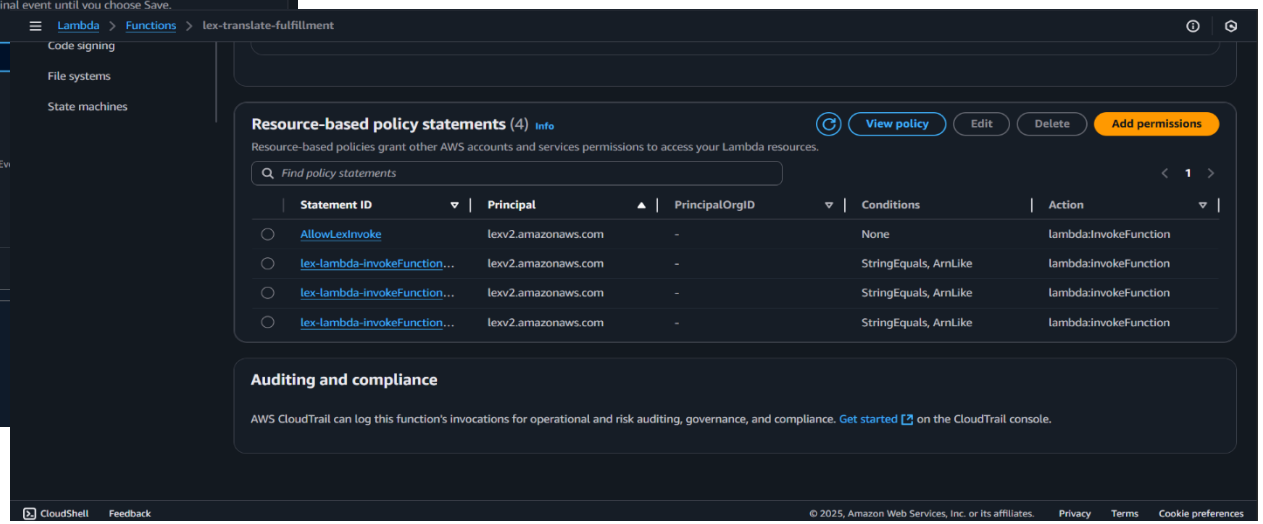


Amazon Lambda- Phase 1

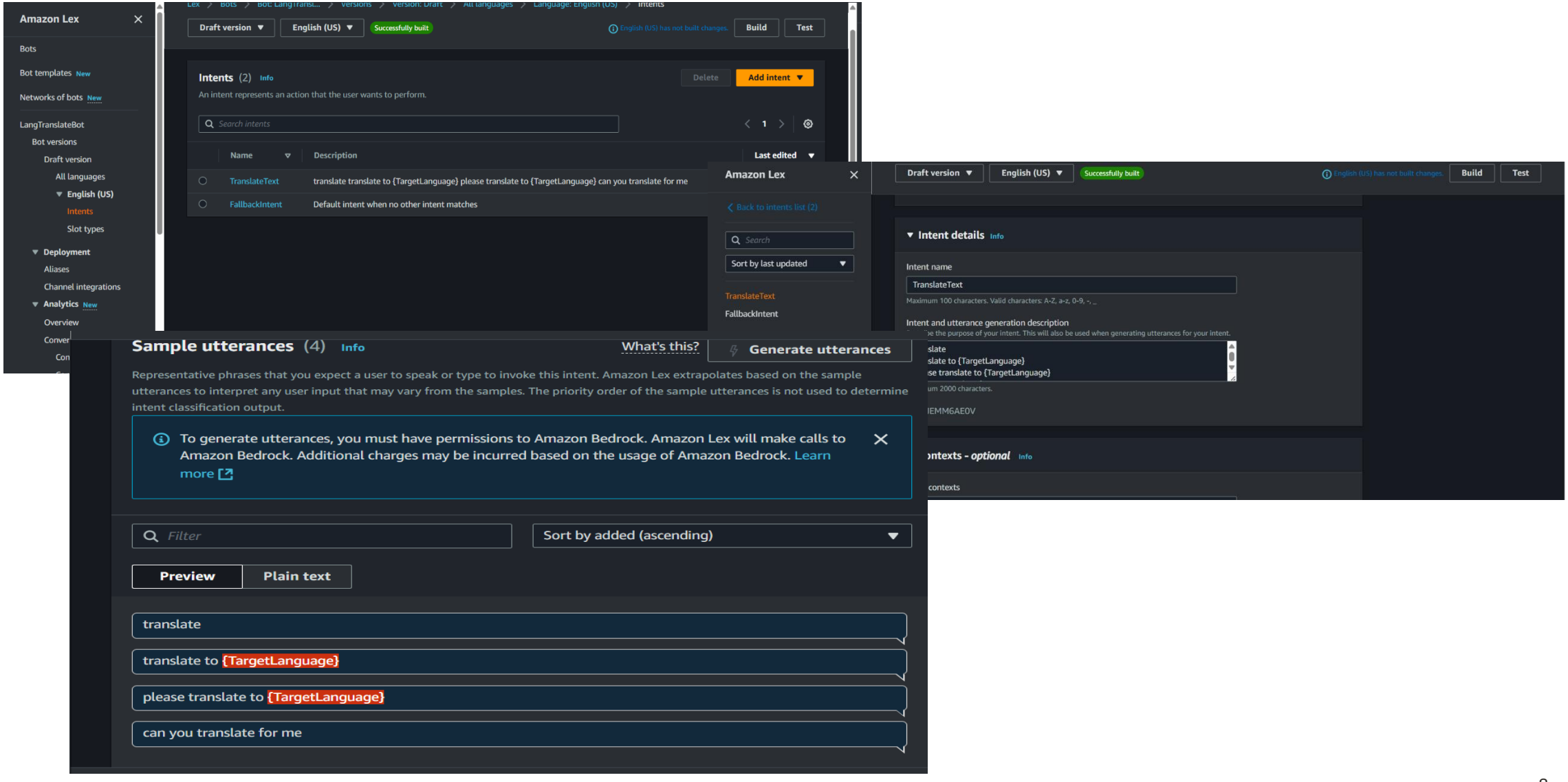




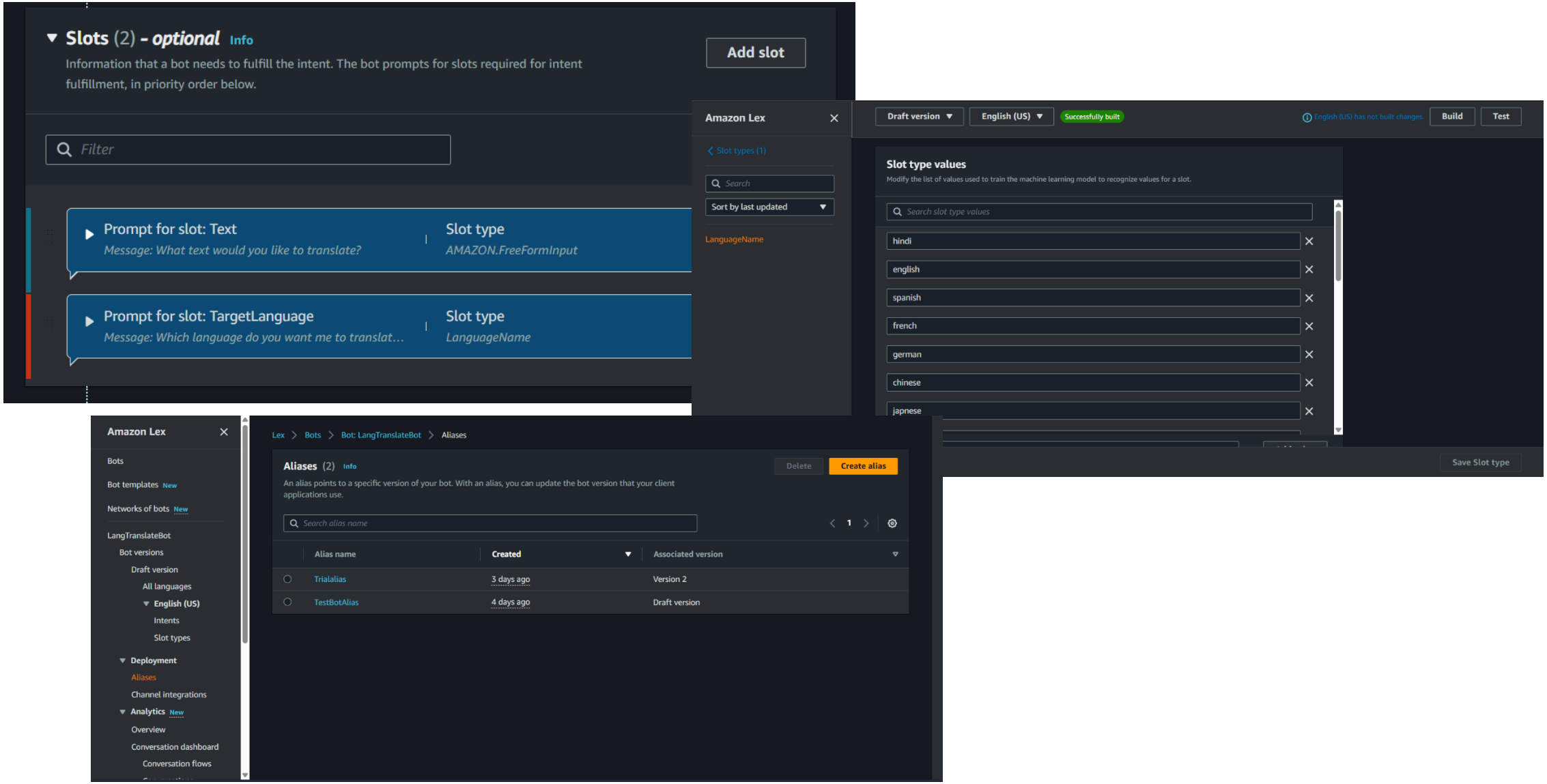




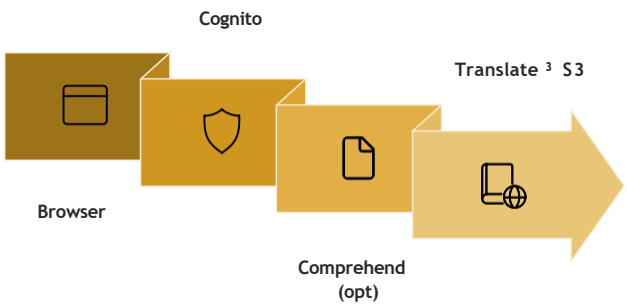
Amazon Lex- Bots, Intents, utterances



Slots and Aliases



Architecture: Phase 2 (Direct Web Translator)



Workflow:

Access Web Translator

User accesses the web translator application hosted on **Amazon S3**.

Obtain Credentials

Browser obtains temporary credentials from **Amazon Cognito** for secure access.

Detect Language

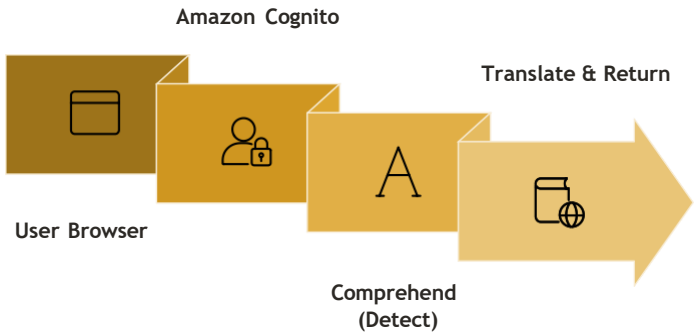
Comprehend detects the input language if the "Auto-Detect" option is selected by the user.

Process Translation

Translate processes the translation request directly from the frontend of the application.

Architecture: Phase 2 - Flow Diagram & Advantages

Flow Diagram



Advantages

No Backend Components

Eliminates the need for traditional backend servers like Lambda or Lex

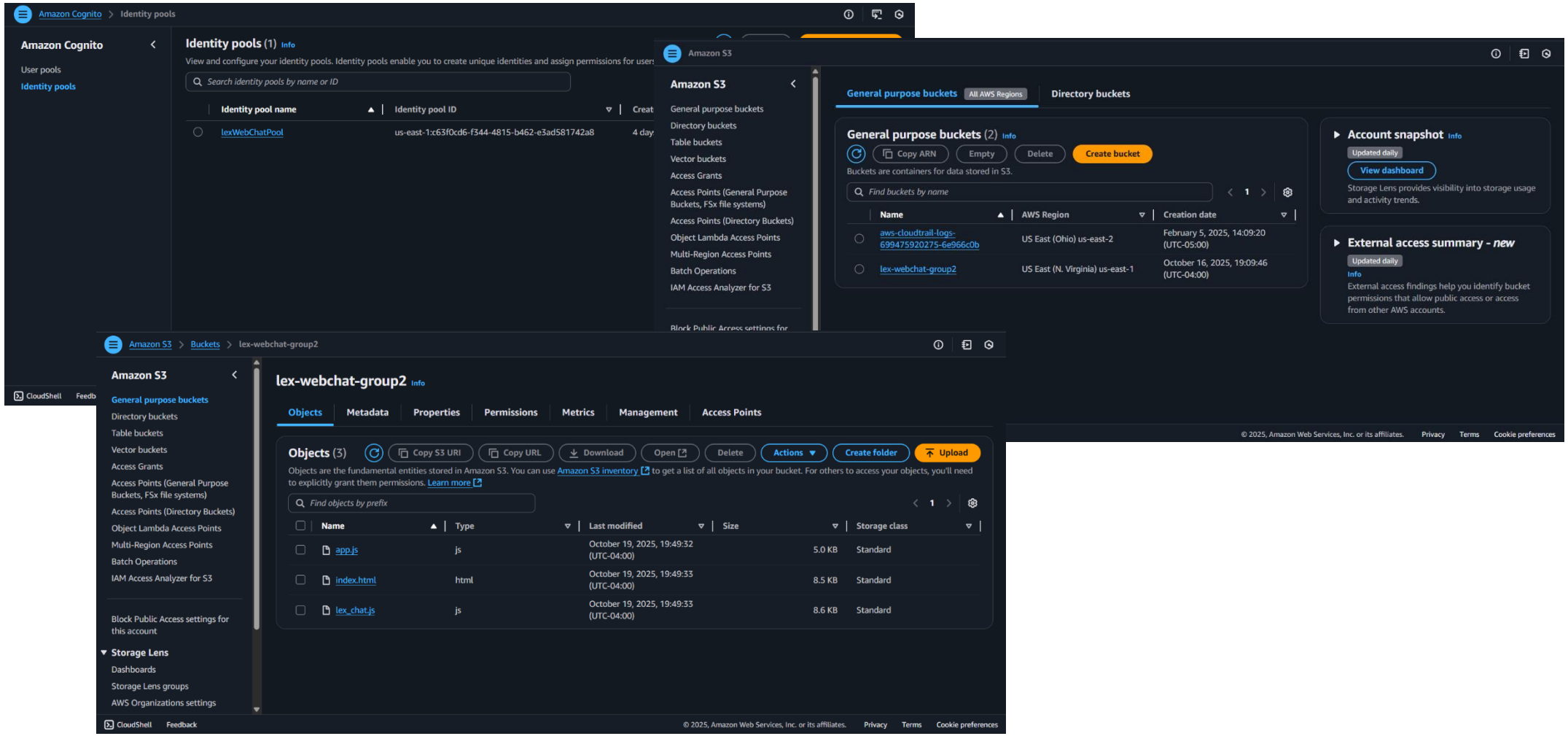
Faster Response & Lower Costs

Achieves quicker response times and reduced operational expenses

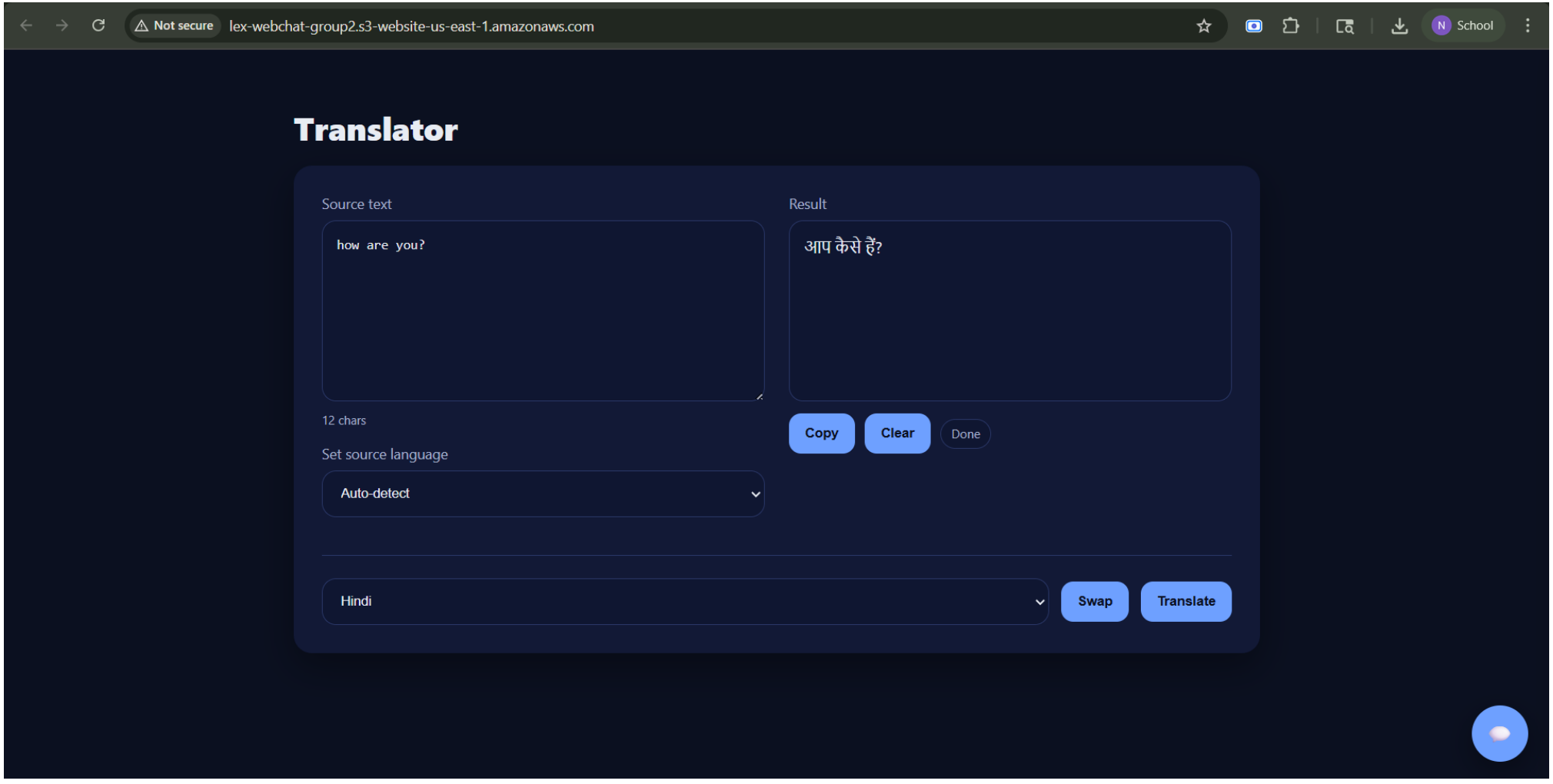
Simplified Deployment

Streamlined deployment under AWS serverless architecture principles

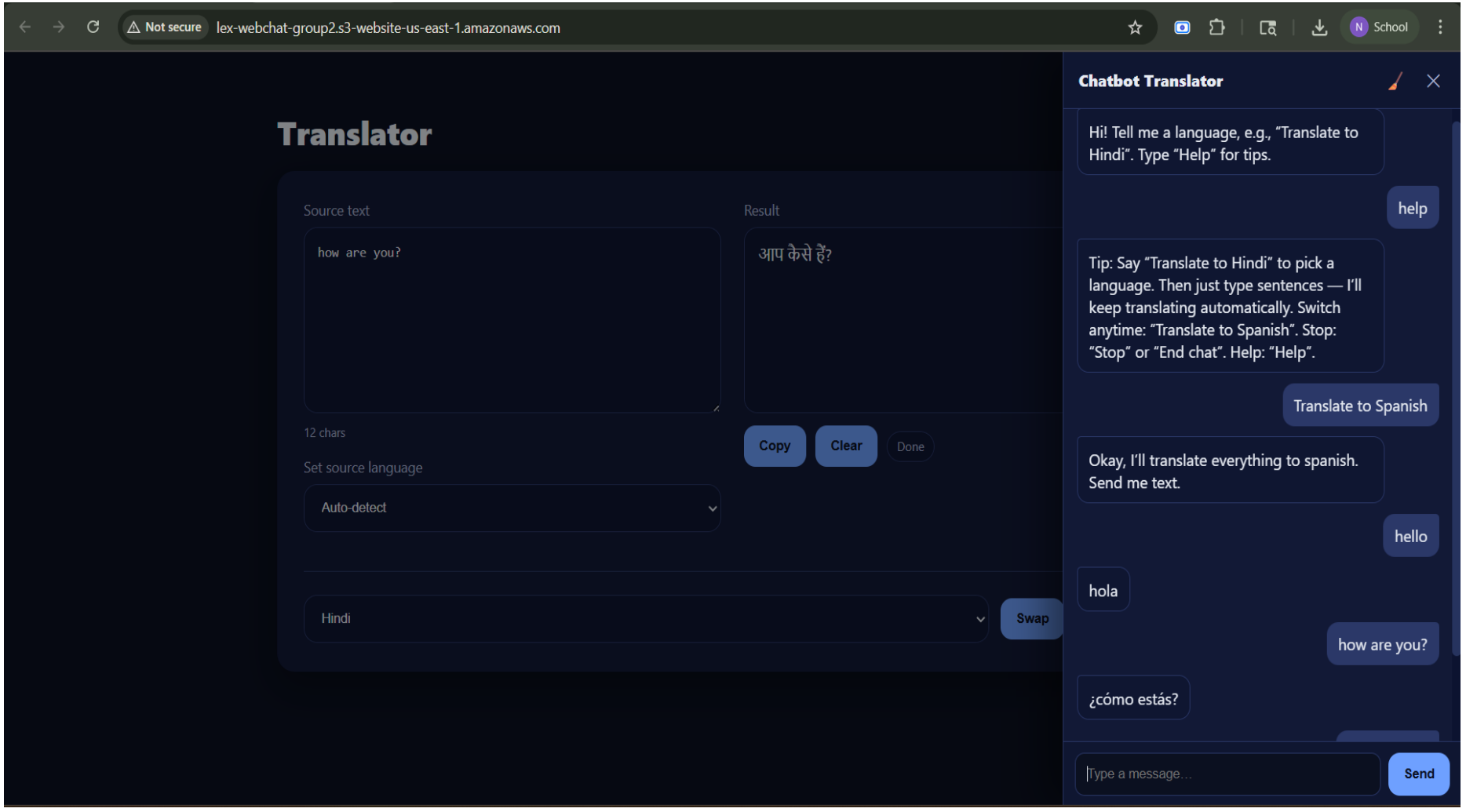
Amazon Cognito and Amazon S3



Translator UI- Default (Phase 2)



Translator UI- Chatbot Interaction (Phase 1)



Aspect	Phase 1 3 Chatbot Version	Phase 2 3 Direct Web Translator
Architecture Type	Serverless backend-driven (via Lex + Lambda)	Client-side AWS integration (browser directly calls AWS services)
Main Entry Point	Amazon Lex V2 Bot (chat interface)	Web Application UI hosted on Amazon S3
Processing Logic	Translation handled inside AWS Lambda using boto3	Translation handled directly in browser using AWS SDK for JavaScript
Language Detection	Performed in Lambda via Amazon Comprehend	Performed optionally via Comprehend API from frontend
Authentication	Managed internally by Lex & Lambda role permissions	Managed through Amazon Cognito Identity Pool issuing temporary credentials
User Interaction Model	Conversational chatbot (<Translate hello to Frenchî)	Direct form-style or text-box input interface
Latency	Slightly higher 4 multiple AWS service hops (Lex ³ Lambda ³ Translate)	Lower 4 direct call to Translate, fewer hops
Deployment Complexity	Higher 4 requires bot configuration, Lambda linking, and permissions	Lower 4 purely static web app + Cognito identity pool
Use Case Fit	Best for interactive, text chatbot experiences	Best for quick, lightweight text translation tools

Challenges and Solutions

IAM Access Denied

Challenge: IAM access denied errors
Resolution: Updated Cognito role policies with Translate and Comprehend permissions.

Missing Language Parameters

Challenge: Missing language parameters
Resolution: Implemented "auto-detect" logic using Comprehend.

Region Configuration

Challenge: Region configuration issues
Resolution: Standardized AWS SDK region to us-east-1.

Cached Outdated Files

Challenge: Cached outdated files on S3
Resolution: Re-uploaded and performed cache refresh.

Complex Lex-Lambda Setup

Challenge: Complex Lex-Lambda setup
Resolution: Simplified architecture to a direct frontend integration.

Results, Key Learnings, and Conclusion

Key Results

- Supports **12+ languages** with auto-detection.
- Achieved **instant translation** and zero backend latency.
- Operates fully on **serverless infrastructure**.
- Static web app deployment using **Amazon S3** hosting.

Key Learnings

- Effective integration of **AI and serverless services** for real-world use cases.
- Importance of **role-based access management** using IAM and Cognito.
- Optimizing architecture reduces **latency and operational costs**.

The AWS Translator Web Application exemplifies how **AWS AI and Serverless technologies** can be combined to deliver scalable, cost-efficient, and intelligent multilingual translation solutions.